



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Clean Code

ALUMNO

Pérez Takayanagui Louis Emilio
322311419

MATERIA

Ingeniería de Software

9 de mayo de 2026

1. Introducción

El video presenta una perspectiva fundamental sobre la ingeniería de software: la eficiencia de un desarrollador no radica en la velocidad de escritura, sino en la legibilidad del código. Basándose en la premisa de que la proporción entre tiempo de lectura y escritura es de 10 a 1, el ponente argumenta que el “código limpio” no es un lujo, sino una necesidad económica y profesional para evitar la acumulación de deuda técnica.

2. Principios fundamentales

- **Responsabilidad Profesional:** El desarrollador es el máximo responsable de la calidad del producto. Al igual que otras profesiones reguladas, no se debe comprometer la integridad técnica ante presiones externas o plazos de entrega poco realistas.
- **Semántica y Nomenclatura:** Se enfatiza el uso de nombres significativos para variables y funciones. El código debe leerse como “prosa bien escrita”, eliminando la necesidad de traducción mental por parte del lector.
- **Código Autodocumentado:** Los comentarios deben ser el último recurso. Un código limpio expresa su intención mediante su estructura y nombres; los comentarios suelen quedar obsoletos y pueden inducir a error a futuros desarrolladores.
- **La Regla del *Boy Scout*:** El principio de mejora continua sugiere dejar el código en mejores condiciones de las que se encontró. Esto combate la degradación natural del software (*bit rot*) mediante refactorizaciones pequeñas pero constantes.
- **Principio de Responsabilidad Única (SRP):** Una función o clase debe tener una sola razón para cambiar. La alta cohesión y el bajo acoplamiento son claves para un sistema mantenible y fácil de testear.
- **Arquitectura Independiente:** Los marcos de trabajo (*frameworks*) son herramientas, no arquitecturas. El diseño del software debe centrarse en los casos de uso y no quedar supeditado a las limitaciones o estructuras de una tecnología específica.
- **Práctica Deliberada:** La maestría en la programación requiere práctica fuera del entorno laboral. El uso de “*katas*” y el estudio de nuevas técnicas permiten al profesional ejecutar con precisión cuando los proyectos son críticos.

3. Conclusión

La sesión concluye que el desarrollo de software de alta calidad requiere un compromiso con estándares éticos y técnicos. Adoptar estas prácticas no solo mejora la mantenibilidad del sistema, sino que dignifica la labor del programador como un profesional técnico riguroso.